MULUNGUSHI UNIVERSITY

SCHOOL OF ENGEERING AND TECHNOLOGY

COURSE NAME:DATA STRUCTURES AND ALGORITHMS

COURSE CODE: ICT 202

TOPIC: STACKS

ASSIGNMENT NUMBER:3

LECTURER: MR KALUBA

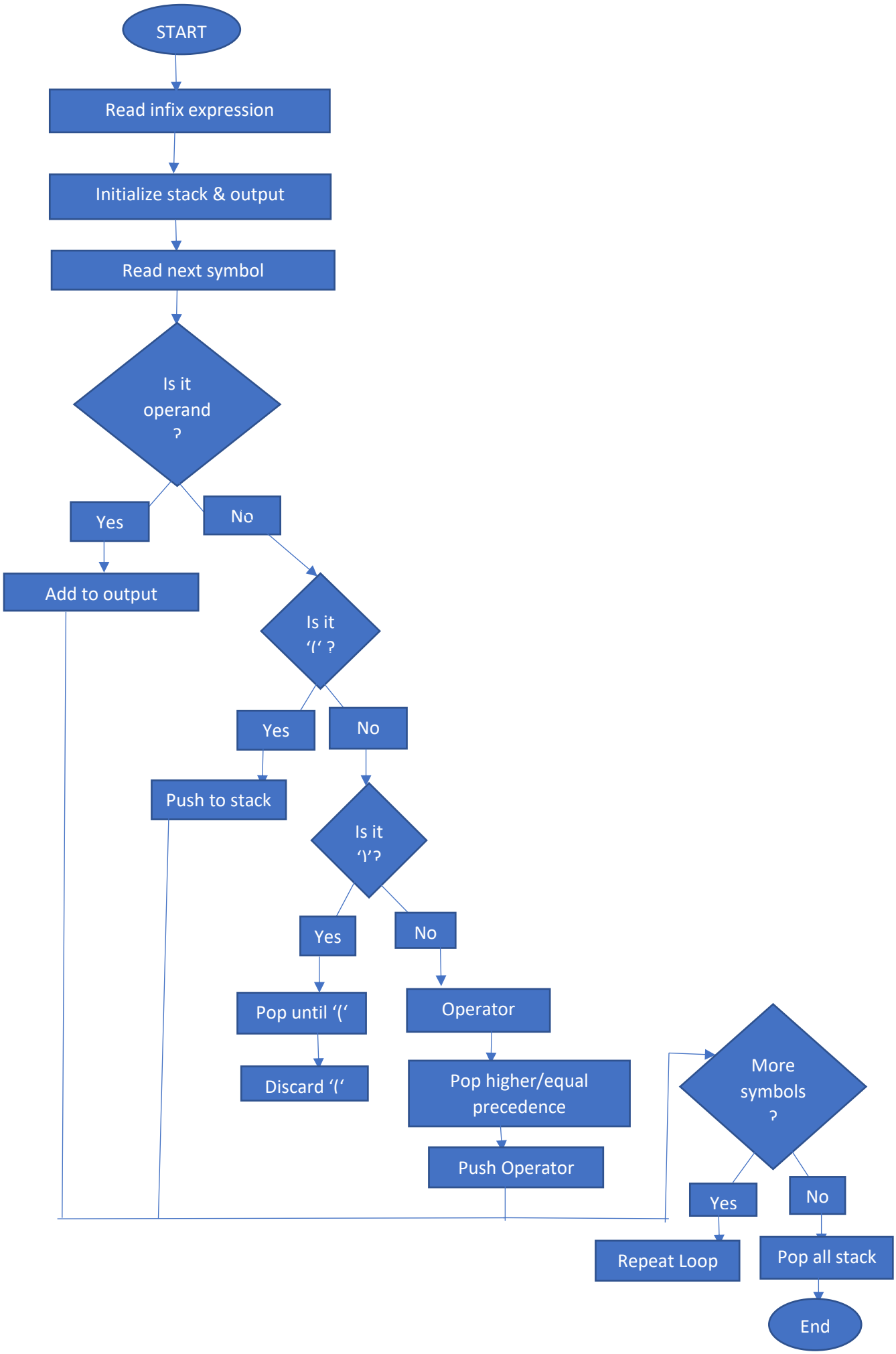
DUE DATE: 16TH APRIL 2026

FLOWCHARTS

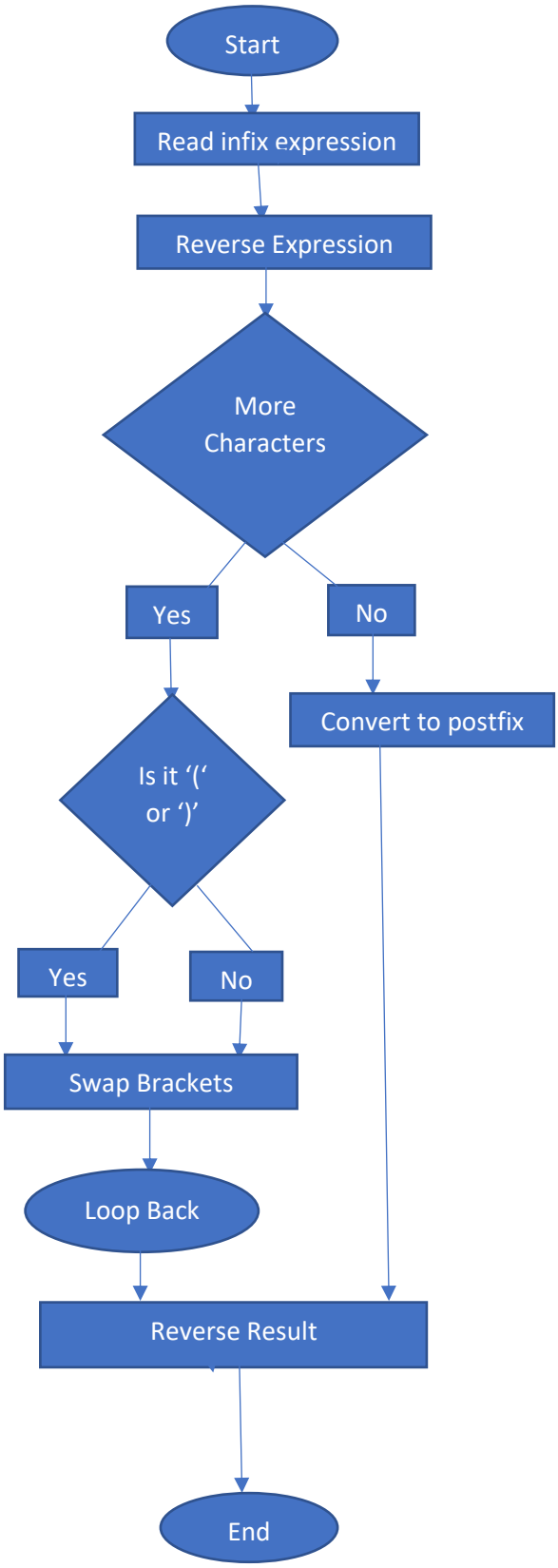
INFIX TO POSTFIX

FLOWCHART

INFIX TO POSTFIX


START
Read infix expression
Initialize stack & output
Read next symbol
Is it operand ?
Yes
No
Add to output
Is it '(' ?
Yes
No
Push to stack
Is it ')'?
Yes
No
Pop until '('
Discard '('
Operator
Pop higher/equal precedence
Push Operator
More symbols ?
Yes
No
Repeat Loop
Pop all stack
End

INFIX TO PREFIX



PSEUDO CODES

1.INFIX TO POSTFIX

```
function infixToPostfix(expression):
   create empty stack
   result = ""
   for each character ch in expression:
      if ch is operand:
         add ch to result
      else if ch == '(':
         push ch to stack
         else if ch == ')':
         while top of stack != '(':
            pop and add to result
         pop '(' from stack
      else:
         while stack not empty AND precedence(top) >= precedence(ch):
            pop and add to result
         push ch to stack
   while stack not empty:
      pop and add to result
   return resultfunction infixToPrefix(expression):
   reverse expression
   for each character:
      if '(' replace with ')'
      if ')' replace with '('
  postfix = infixToPostfix(reversed expression)
```

```
  reverse postfix
   return prefix
```

2.<u>INFIX TO PREFIX</u>

```
function infixToPrefix(expression):
   reverse expression
   for each character:
      if '(' replace with ')'
      if ')' replace with '('
   postfix = infixToPostfix(reversed expression)
   reverse postfix
   return prefix
```

JAVA PROGRAM

```
import java.util.Stack;
public class InfixConversion {
  // Check precedence
   static int precedence(char ch) {
      if (ch == '+' || ch == '-') return 1;
      if (ch == '*' || ch == '/') return 2;
      return -1;
   }

   // Check operand
   static boolean isOperand(char ch) {
```

```
    return Character.isLetterOrDigit(ch);
}

// Infix to Postfix
static String infixToPostfix(String exp) {
    StringBuilder result = new StringBuilder();
    Stack<Character> stack = new Stack<>();

    for (char ch : exp.toCharArray()) {

        if (isOperand(ch)) {
            result.append(ch);
        }

        else if (ch == '(') {
            stack.push(ch);
        }

        else if (ch == ')') {
            while (stack.peek() != '(') {
                result.append(stack.pop());
            }
            stack.pop();
        }

        else {
```

```
            while (!stack.isEmpty() &&
                precedence(stack.peek()) >= precedence(ch)) {
                result.append(stack.pop());
            }
            stack.push(ch);
        }
    }

    while (!stack.isEmpty()) {
        result.append(stack.pop());
    }

    return result.toString();
}

// Reverse string
static String reverse(String str) {
    return new StringBuilder(str).reverse().toString();
}

// Infix to Prefix
static String infixToPrefix(String exp) {
    // Step 1: Reverse
    String reversed = reverse(exp);

    // Step 2: Swap brackets
```

```
        StringBuilder modified = new StringBuilder();
        for (char ch : reversed.toCharArray()) {
            if (ch == '(')
                modified.append(')');
            else if (ch == ')')
                modified.append('(');
            else
                modified.append(ch);
        }

        // Step 3: Convert to postfix
        String postfix = infixToPostfix(modified.toString());

        // Step 4: Reverse postfix
        return reverse(postfix);
    }

    public static void main(String[] args) {
        String infix = "A+B*C-D/F";

        System.out.println("Infix: " + infix);
        System.out.println("Postfix: " + infixToPostfix(infix));
        System.out.println("Prefix: " + infixToPrefix(infix));
    }
}
```

```
C:\WINDOWS\system32\cmd.

Microsoft Windows [Version 10.0.26100.8117]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HC ELETRONICS>CD DESKTOP

C:\Users\HC ELETRONICS\Desktop>javac InfixConversion.java

C:\Users\HC ELETRONICS\Desktop>java InfixConversion
Infix: A+B*C-D/F
Postfix: ABC*+DF/-
Prefix: +A-*BC/DF

C:\Users\HC ELETRONICS\Desktop>
```